

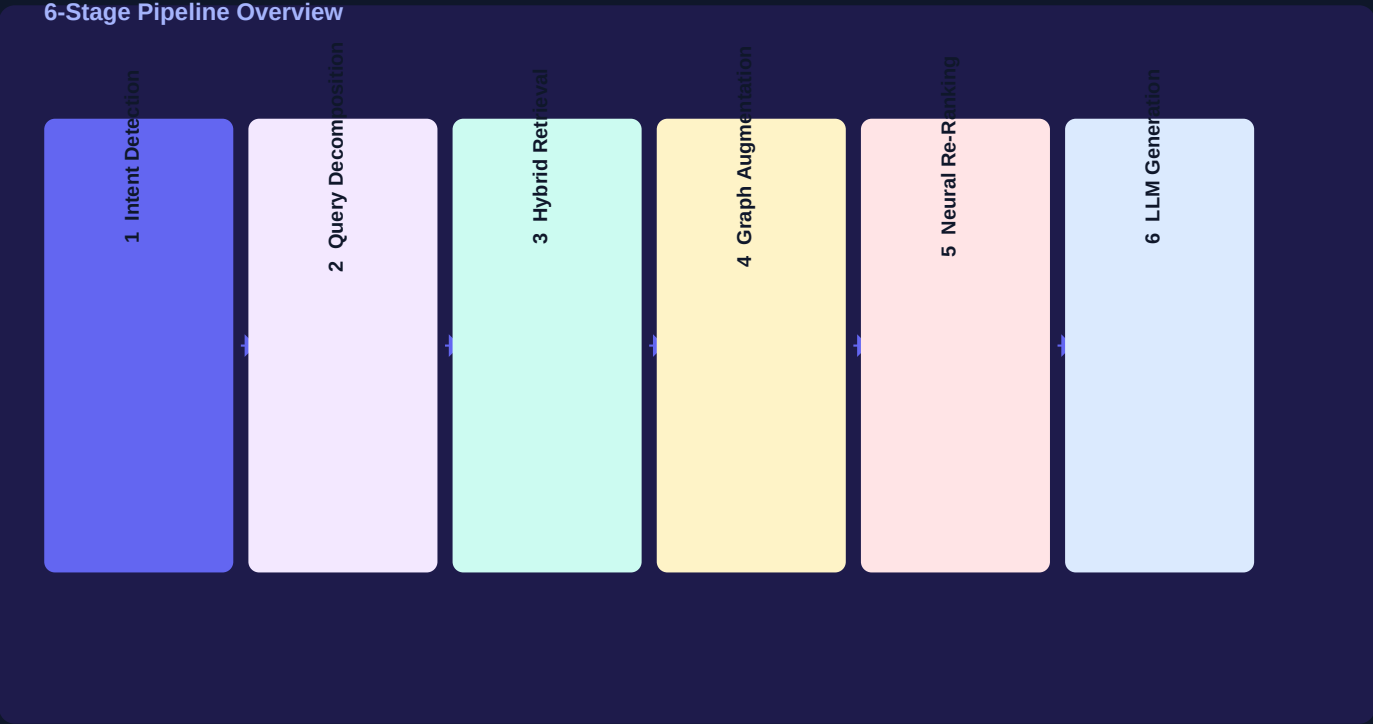
PRISM

Intent-Aware & Explainable Hybrid Retrieval System

HCL Hack60 — 30-Day Project Plan with Pipeline & Testing Checkpoints

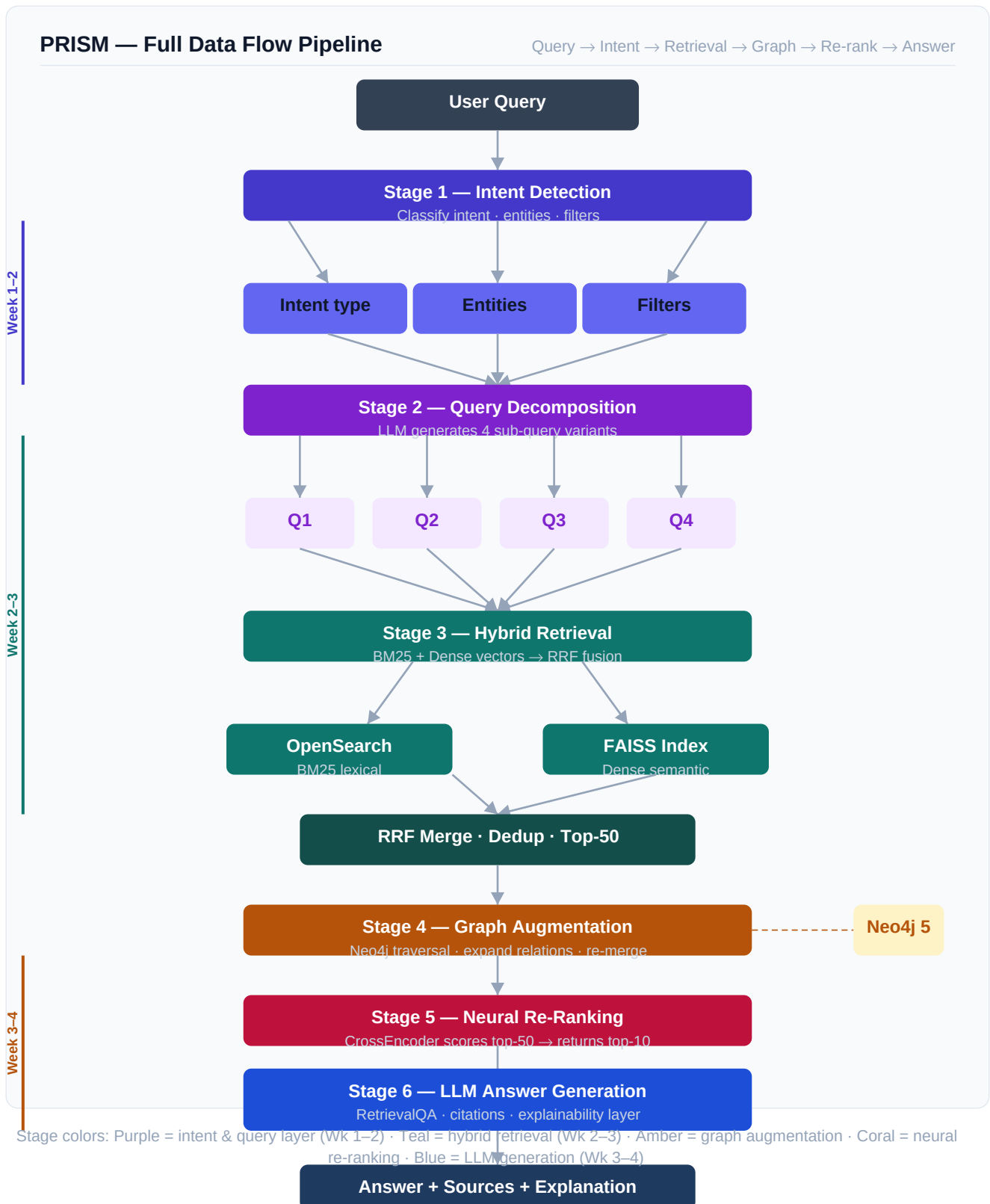
- Python 3.10
- LangChain
- FAISS
- Neo4j
- FastAPI
- RAGAS
- Ollama

DURATION	FINAL OUTPUT	LLM BACKEND	EVALUATION
30 Days / 4 Weeks	Web App + REST API + KG Ollama / GPT-4o		RAGAS Framework



Data Flow Pipeline

Every user query flows through six sequential stages before an answer is returned. Each stage is independently testable and maps to a specific week in the 30-day plan.



Submission Target Metrics

≥ 95%
Retrieval Rate

> 0.80
Context Recall

> 0.75
Faithfulness

> 0.75
Answer Relevancy

< 10s
API Response

+10% P@5
Re-rank Gain

Technology Stack

Layer	Technology / Library
Embeddings	sentence-transformers (all-MiniLM-L6-v2 / all-mpnet-base-v2)
Vector Store	FAISS (dense) + OpenSearch BM25 (sparse)
LLM Orchestration	LangChain — chains, retrievers, LCEL, memory, prompts
LLM Backend	Ollama + Mistral 7B (local default) / OpenAI GPT-4o
Knowledge Graph	Neo4j 5 + neo4j Python driver + GraphCypherQChain
Re-Ranking	HuggingFace CrossEncoder (ms-marco-MiniLM-L-6-v2)
Evaluation	RAGAS — context_precision, context_recall, faithfulness, relevancy
API Layer	FastAPI + Pydantic + Uvicorn (async)
Frontend	React + Tailwind (preferred) / Streamlit (fast)
Containerization	Docker + docker-compose (OpenSearch & Neo4j)
Dataset	Indian Legal Docs — indiankanoon.org / Kaggle SC Judgments

WEEK 1 — Foundation & Data Pipeline

Days 1–7

Day 1–2 Environment Setup

- Create venv, install langchain, sentence-transformers, faiss-cpu, opensearch-py, neo4j
- Start Docker: OpenSearch on :9200, Neo4j on :7474/:7687
- Create project structure: /data /embeddings /retrieval /reranker /graph /api /eval
- Set .env file: OPENAI_API_KEY, NEO4J_PASSWORD, OPENSEARCH_HOST

```
import langchain, faiss, sentence_transformers
print("All imports OK")
```

TESTING CHECKPOINTS

- ✓ `python -c "import langchain; print(langchain.__version__)"` prints version
- ✓ `curl http://localhost:9200` returns JSON cluster info
- ✓ `http://localhost:7474` loads Neo4j Browser

Day 3–4 Dataset Selection & Preprocessing

- Download: indianoon.org OR Kaggle Supreme Court Judgments dataset
- Clean text: strip HTML tags, fix UTF-8, remove boilerplate headers/footers
- Chunk with RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)
- Store chunks in SQLite: id, source, page, chunk_text, metadata_json

TESTING CHECKPOINTS

- ✓ Total chunks > 1000 for meaningful retrieval testing
- ✓ No chunk is empty or contains raw HTML / \x escapes
- ✓ `SELECT COUNT(*) FROM chunks` matches expected number

Day 5–6

Embedding Pipeline

- Load all-MiniLM-L6-v2 (fast) or all-mpnet-base-v2 (accurate)
- Batch-encode chunks (batch_size=64), save embeddings.npy
- Build FAISS IndexFlatL2(384), add vectors, save index to disk
- Bulk-index chunk texts in OpenSearch with BM25 analyzer

```
index = faiss.IndexFlatL2(384)
index.add(embeddings)
D, I = index.search(query_vec, k=5)
```

TESTING CHECKPOINTS

- ✓ embeddings.shape == (num_chunks, 384)
- ✓ FAISS query 'legal judgment privacy' returns top-5 in < 200ms
- ✓ OpenSearch BM25 top-5 overlaps partially with FAISS results

WEEK 2 — Core Retrieval System

Days 8–14

Day 8–9 Hybrid Search with RRF Fusion

- Wrap FAISS in LangChain FAISS vectorstore + InMemoryDocstore
- Wrap OpenSearch as LangChain retriever (OpenSearchVectorSearch)
- Implement RRF: $\text{score} = \sum(1 / (k + \text{rank}_i))$ where $k=60$
- EnsembleRetriever(weights=[0.5, 0.5]) — test on 20 sample queries

```
retriever = EnsembleRetriever(  
    retrievers=[bm25, faiss], weights=[0.5, 0.5])
```

TESTING CHECKPOINTS

- ✓ Hybrid top-3 outperforms BM25-only and FAISS-only on same query
- ✓ Results are merged and deduplicated (no duplicate doc IDs)
- ✓ $\text{len}(\text{hybrid_results}) \geq \max(\text{len}(\text{bm25}), \text{len}(\text{faiss}))$

Day 10–11 Intent Detection & Query Decomposition

- PromptTemplate chain for intent: factual / exploratory / navigational
- Entity extraction chain: date, domain, entity type → structured dict
- Query decomposition: one query → 2–4 sub-queries via LLM
- Use Ollama + Mistral 7B locally (ollama pull mistral)

TESTING CHECKPOINTS

- ✓ 'What is Section 377?' → intent: factual
- ✓ 'Recent Supreme Court cases 2023' → {year: 2023, domain: legal}
- ✓ Output is valid Python dict — no JSON parse errors

Day 12–13

Multi-Query Retrieval

- MultiQueryRetriever wrapping EnsembleRetriever, 4 variants per query
- Merge/dedup all variant results using content hash (UniqueDocumentFilter)
- Compare Recall@10 before vs after multi-query — expect +15–25%

TESTING CHECKPOINTS

- ✓ 4 generated variants are semantically different from each other
- ✓ No duplicate documents in final result list
- ✓ Full multi-query completes in < 3 seconds

Day 14

Benchmarking Round 1

- Create 50-query test set with known relevant documents (ground truth)
- Compute Precision@5, Precision@10, Recall@10 — log to CSV
- Identify bottom-10 worst queries for targeted improvement

TESTING CHECKPOINTS

- ✓ Precision@5 > 0.60 — at least 3 of top 5 results are relevant
- ✓ Recall@10 > 0.70
- ✓ Git tag: v0.2-hybrid-retrieval committed

WEEK 3 — Knowledge Graph + Re-Ranking + Full RAG

Days 15–21

Day 15–16 Knowledge Graph with Neo4j

- Schema: Nodes (Document, Person, Case, Law, Date), Edges (MENTIONS, RELATED_TO, CITES)
- Extract named entities using spaCy en_core_web_sm on each chunk
- Ingest with MERGE (never CREATE) to prevent duplicate nodes
- GraphCypherQACChain for natural-language → Cypher translation

```
s.run('MERGE (d:Document {id: $id})', id='doc_001')
```

TESTING CHECKPOINTS

- ✓ MATCH (n) RETURN count(n) shows > 500 nodes in Neo4j Browser
- ✓ MATCH ()-[r]->>() RETURN count(r) shows > 1000 relationships
- ✓ Re-running ingestion does NOT create duplicate nodes

Day 17 Graph-Augmented Retrieval

- Extract entities from top-5 vector results, query Neo4j for RELATED_TO / CITES
- Merge graph-found docs with vector results via RRF (graph weight 0.3)
- Pipeline: Vector → Graph Expansion → RRF Merge

TESTING CHECKPOINTS

- ✓ Graph-expanded results include >= 1 doc not in pure vector results
- ✓ Pipeline doesn't crash when Neo4j returns 0 related nodes
- ✓ Augmented retrieval completes in < 5 seconds end-to-end

Day 18–19 Neural Re-Ranking with Cross-Encoder

- Load cross-encoder/ms-marco-MiniLM-L-6-v2 from HuggingFace
- rerank(query, docs, top_k=10) using cross-encoder scores
- Pipeline: Hybrid top-50 → Cross-Encoder → top-10 returned to user

```
scores = ce.predict([(query, doc.page_content) for doc in docs])
```

TESTING CHECKPOINTS

- ✓ Avg cross-encoder score of re-ranked top-10 > original top-10
- ✓ Re-ranking 50 docs completes in < 2 seconds on CPU
- ✓ Top-10 is a strict subset of top-50 (no new docs introduced)

Day 20–21 Full RAG Pipeline Assembly

- Chain: Intent → Multi-Query → Hybrid → Graph → Re-rank → LLM
- RetrievalQA with return_source_documents=True for citations
- ConversationBufferWindowMemory (last 5 turns) for multi-turn context
- Answer post-processing: format, clean, add confidence indicator

TESTING CHECKPOINTS

- ✓ 10 test questions all return coherent answers with source citations
- ✓ Follow-up question maintains context from previous turn
- ✓ Out-of-scope question returns 'Not found' gracefully
- ✓ Answer generation time < 15 seconds including all retrieval steps

WEEK 4 — Explainability + Evaluation + API / UI

Days 22–30

Day 22–23 Explainability Layer

- Per-result 'Why retrieved?' explanation via LLM + retrieval metadata
- Source tag per result: BM25 / Semantic / Graph with % contribution
- Confidence breakdown: BM25 score, cosine similarity, cross-encoder score
- Highlight matching terms (token overlap between query and result)

TESTING CHECKPOINTS

- ✓ Every result has non-empty 'explanation' field
- ✓ BM25-found doc does NOT show 'Semantic' source tag
- ✓ All explanations grammatically correct across 10 test queries

Day 24–25 RAGAS Evaluation Suite

- Auto-generate 100 (question, ground_truth, contexts) triplets using LLM
- Run RAGAS: context_precision, context_recall, answer_relevancy, faithfulness
- Save eval_results.csv — iterate until thresholds met

```
results = evaluate(dataset, metrics=[context_precision,
                                     context_recall, answer_relevancy, faithfulness])
```

TESTING CHECKPOINTS

- ✓ Context Recall > 0.80, Faithfulness > 0.75, Answer Relevancy > 0.75
- ✓ Bottom-10 worst queries identified and at least 5 fixed via tuning
- ✓ Final RAGAS scores >= baseline from before tuning

Day 26–27**FastAPI Backend**

- POST /search: {query, filters} → {results, explanations}
- GET /explain/{doc_id}: full explanation for a result
- GET /graph/{entity}: Neo4j neighbourhood of an entity
- POST /feedback: thumbs-up/down for future fine-tuning
- Pydantic input validation + HTTPException error handling

TESTING CHECKPOINTS

- ✓ POST /search returns JSON in < 10s via curl / Postman
- ✓ 5 concurrent requests all complete successfully (load test)
- ✓ http://localhost:8000/docs shows all endpoints in Swagger UI

Day 28–29**Web App Frontend**

- React + Tailwind (preferred) or Streamlit (fast)
- Search bar with real-time suggestions, result snippets, explanation badges
- Filter sidebar: date range, domain, document type
- Knowledge graph visualisation: vis.js or react-force-graph

TESTING CHECKPOINTS

- ✓ Search returns results in < 10s and displays correctly
- ✓ Filtering by year shows only docs from that year
- ✓ Graph renders without JS errors in browser console

Day 30**Final Testing, Polish & Submission**

- Run 20 fresh queries — verify 95%+ retrieval rate (19/20 relevant)
- Write README: overview, setup, architecture diagram, how to run
- Record 3–5 min demo video: search, explanation, graph visualisation
- Package: requirements.txt + docker-compose.yml + sample data

TESTING CHECKPOINTS

- ✓ docker-compose up brings entire system up from scratch
- ✓ All RAGAS metrics meet minimum thresholds on final evaluation run
- ✓ GitHub repo is public and README runnable by a stranger
- ✓ Git tag: v1.0-submission committed

Pro Tips for Success

★ Use Ollama locally Run 'ollama pull mistral' to avoid OpenAI API costs during all development phases.	★ Cache embeddings Save to .npy files immediately — recomputing wastes hours on every restart.
★ Start RAGAS early Run evaluation on small batches from Week 2, not just at the end.	★ Daily log Keep daily_log.md — note what worked, what failed, and benchmark scores each day.
★ Use MERGE not CREATE For Neo4j ingestion always use MERGE to prevent duplicate node explosions.	★ top-k strategy Set retrieval top-k=50 for cross-encoder input but return only top-10 to the user.
★ Test edge cases Empty query, very long query (>200 words), queries in Hindi/mixed language.	★ Use LCEL Use LangChain's LCEL pipe operator for cleaner chain composition from Week 3 onward.
★ Fix LLMChain LLMChain is deprecated — use 'prompt llm parser' LCEL syntax in new LangChain.	★ Secrets management Never hardcode keys. Use python-dotenv and a .env file from Day 1.